



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

**Dept. of Computer Science
Faculty of Science and Technology**

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Spring 2025-2026

Section: [I]

Group No: 08

Project Report On

Street Children Care System

Supervised By

Tofayet Sultan

Name	Id
ISHMAM MUBTASIM	23-51771-2

Submitted By:

ISHMAM MUBTASIM

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
--------------------------------	--	-----------------------------	------------------------	---------------------	--------------------------

Evaluation Criteria	Evaluation Definition	Total =
Requirement fulfillment	Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.	
Validation	Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.	
Verification	Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.	

TABLE OF CONTENTS

No.	Section Title	Page No.
1	Introduction	04
2	Case Study / Scenario	05
3	ER Diagram	06
4	Normalization	07
5	Finalization	08
6	Database Schema Creation	09
7	All Tables	11
8	User Story	14
9	System Modules	16
10	Screenshots of the Application	17
11	Schema Diagram	26
12	Implementation	27
13	Conclusion	64
14	GitHub Link	64
15	References	65

INTRODUCTION

Context & Problem

Bangladesh has a large number of street children who lack stable shelter, proper identification, and consistent healthcare. Organizations such as orphanages, NGOs, and charitable foundations work to register and support these children, but most still rely on paper-based, manual record-keeping methods. These systems cause data loss, duplication, missed vaccination schedules, and uncontrolled access to sensitive records. Without a centralized digital solution, managing child welfare data accurately and securely remains a serious challenge for these organizations.

The Solution & Tech Stack

The Street Children Care System is a desktop application built with C# (.NET Windows Forms) in Microsoft Visual Studio, connected to a relational database managed through SQL Server Management Studio (SSMS). The application uses ADO.NET with parameterized SQL queries to ensure fast, secure, and reliable database communication. It provides a centralized platform where authorized users can manage all child-related data through a clean, role-based interface - replacing paper records entirely.

Core Capabilities

- **Role-Based Login-** Admin users get full data management access; Staff users get view and search access only.
- **Children & Institution Management-** Full CRUD operations for Children, Orphanages, Foundations, and Vaccinations (Admin only).
- **Vaccination Tracking-** Each child's vaccination schedule is tracked with Pending, Completed, and Overdue status indicators.
- **Admin Dashboard-** Displays real-time summary counts of all records and flags overdue vaccinations instantly.

Project Objective & Scope

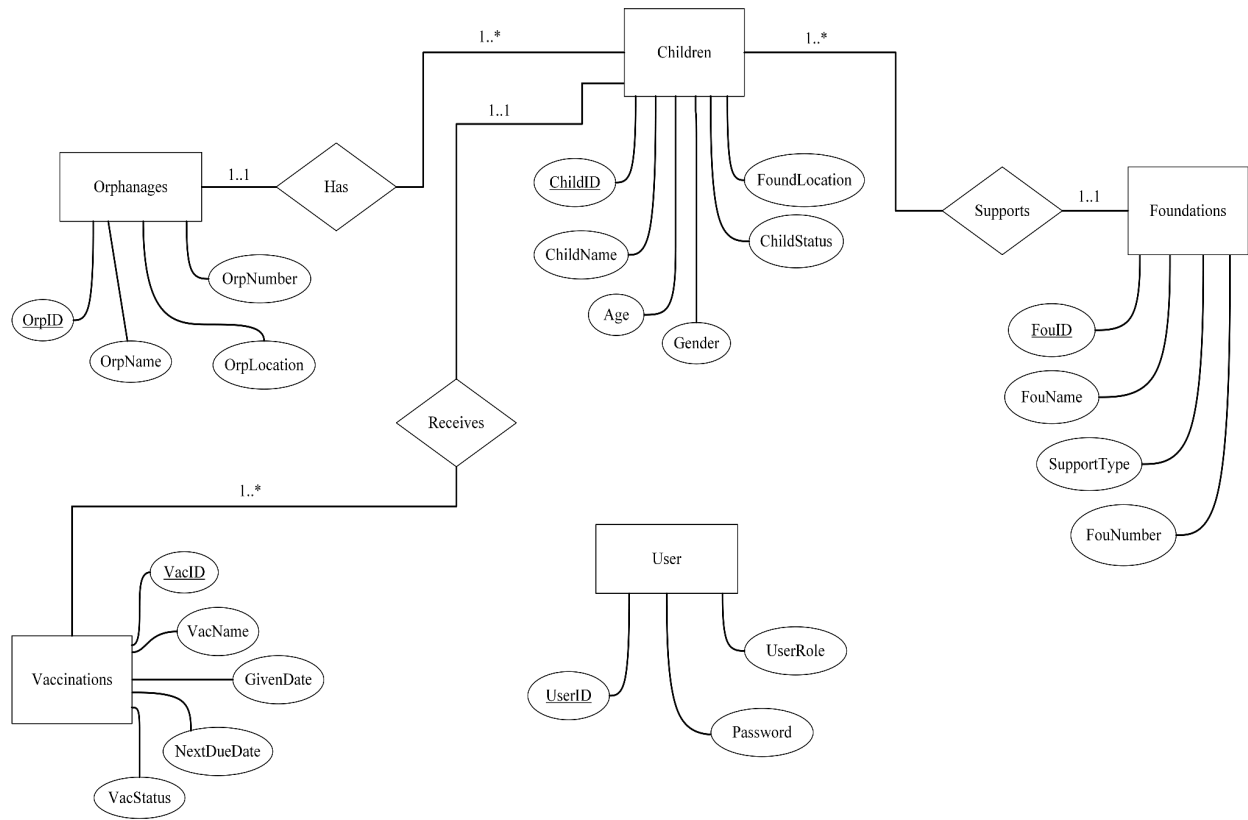
The primary objective of this project is to build a secure, centralized, and role-sensitive management system that tracks street children's registration, institutional placement, and health records in one unified desktop application. The system covers login authentication, full CRUD operations across five database tables, and a dedicated user management module for Admin. It does not include online access, mobile features, or integration with external government databases.

CASE STUDY / SCENARIO

Street Children Care System

In a Street Children Care System, street children can be registered and linked to orphanages and foundations that support them. Each child has a unique ChildID, full name, age, gender, found location, a status which can be Registered, In Orphanage, or Adopted, an OrphanageID as a foreign key, and a FoundationID as a foreign key. Orphanages are identified by an OrphanageID and also store the orphanage name, location, and contact number. One orphanage can have many children, but each child belongs to only one orphanage. This is a One-to-Many relationship between Orphanages and Children. Foundations are identified by a FoundationID and also store the foundation name, support type such as Food, Clothing, Education, or Medical, and a contact number. One foundation can support many children, but each child is linked to only one foundation. This is a One-to-Many relationship between Foundations and Children. Each child can have many vaccination records, but each vaccination record belongs to only one child. This is a One-to-Many relationship between Children and Vaccinations. Every vaccination record has a unique VaccinationID, a ChildID as a foreign key, a vaccine name such as BCG, Polio, or DPT, a given date, a next due date, and a status which can be Pending, Completed, or Overdue. The system also contains a Login module where authorized users can access the system using UserID and Password. Each user has a specific role such as Admin or Staff. Admin users can add, update, save, and delete records, while Staff users can only view information. This system helps to keep track of street children, orphanages, foundations, and vaccinations.

ER DIAGRAM



NORMALIZATION

1. Has: (One to many)

UNF: OrpID, OrpName, OrpLocation, OrpNumber, ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation.

1NF: OrpID, OrpName, OrpLocation, OrpNumber, ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation.

2NF:

1. OrpID, OrpName, OrpLocation, OrpNumber.

2. ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation, OrpID (FK).

3NF: Same as 2NF.

2. Supports: (One to many)

UNF: ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation, FouID, FouName, SupportType, FouNumber.

1NF: ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation, FouID, FouName, SupportType, FouNumber.

2NF:

1. ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation, FouID (FK).

2. FouID, FouName, SupportType, FouNumber.

3NF: Same as 2NF.

3. Receives: (One to many)

UNF: ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation, VacID, VacName, GivenDate, NextDueDate, VacStatus.

1NF: ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation, VacID, VacName, GivenDate, NextDueDate, VacStatus.

2NF:

1. ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation.

2. VacID, VacName, GivenDate, NextDueDate, VacStatus, ChildID (FK).

3NF: Same as 2NF.

4. User Management:

UNF: UserID, Password, UserRole.

1NF: UserID, Password, UserRole.

2NF:

1. UserID, Password, UserRole.

3NF: Same as 2NF.

FINALIZATION

1. OrpID, OrpName, OrpLocation, OrpNumber. (Orphanages)
2. ChildID, ChildName, Age, Gender, ChildStatus, FoundLocation, OrpID (FK), FouID (FK). (Children)
3. FouID, FouName, SupportType, FouNumber. (Foundations)
4. VacID, VacName, GivenDate, NextDueDate, VacStatus, ChildID (FK). (Vaccinations)
5. UserID, Password, UserRole. (User)

DATABASE SCHEMA CREATION

1. SQL command for creating Users Table:

```
CREATE TABLE Users
(
    UserID    NVARCHAR(50) NOT NULL
              PRIMARY KEY,
    Password  NVARCHAR(50) NOT NULL,
    UserRole  NVARCHAR(20) NOT NULL
              CHECK (UserRole IN ('Admin', 'Staff'))
);
```

2. SQL command for creating Orphanages Table:

```
CREATE TABLE Orphanages
(
    OrpID     INT          NOT NULL PRIMARY KEY IDENTITY(1,1),
    OrpName   NVARCHAR(150) NOT NULL,
    OrpLocation NVARCHAR(200) NOT NULL,
    OrpNumber NVARCHAR(20)  NOT NULL
);
```

3. SQL command for creating Foundations Table:

```
CREATE TABLE Foundations
(
    FouID     INT          NOT NULL PRIMARY KEY IDENTITY(1,1),
    FouName   NVARCHAR(150) NOT NULL,
    SupportType NVARCHAR(100) NOT NULL
              CHECK (SupportType IN
                    ('Food', 'Clothing', 'Education', 'Medical')),
    FouNumber NVARCHAR(20)  NOT NULL
);
```

4. SQL command for creating Children Table:

```

CREATE TABLE Children
(
    ChildID    INT        NOT NULL PRIMARY KEY IDENTITY(1,1),
    ChildName  NVARCHAR(100) NOT NULL,
    Age        INT        NOT NULL
                CHECK (Age > 0 AND Age < 18),
    Gender     NVARCHAR(10) NOT NULL
                CHECK (Gender IN ('Male', 'Female')),
    FoundLocation NVARCHAR(200) NOT NULL,
    ChildStatus NVARCHAR(50) NOT NULL
                DEFAULT 'Registered'
                CHECK (ChildStatus IN
                ('Registered', 'In Orphanage', 'Adopted')),
    OrpID      INT        NULL
                FOREIGN KEY REFERENCES Orphanages(OrpID),
    FoulID     INT        NULL
                FOREIGN KEY REFERENCES Foundations(FoulID)
);

```

5. SQL command for creating Vaccinations Table:

```

CREATE TABLE Vaccinations
(
    VacID      INT        NOT NULL PRIMARY KEY IDENTITY(1,1),
    ChildID    INT        NOT NULL
                FOREIGN KEY REFERENCES Children(ChildID),
    VacName    NVARCHAR(100) NOT NULL
                CHECK (VacName IN
                ('BCG', 'Polio', 'DPT', 'Measles', 'Hepatitis B')),
    GivenDate  DATE        NOT NULL,
    NextDueDate DATE      NOT NULL,
    VacStatus  NVARCHAR(50) NOT NULL
                DEFAULT 'Pending'
                CHECK (VacStatus IN ('Pending', 'Completed', 'Overdue'))
);

```

ALL TABLES

1. Users Table:

```
SELECT * FROM StreetChildrenCareDB.dbo.Users;
```

	UserID	Password	UserRole
1	admin01	admin123	Admin
2	admin02	admin456	Admin
3	staff01	staff123	Staff
4	staff02	staff456	Staff
5	staff03	staff789	Staff
6	staff04	staff101112	Staff

2. Orphanages Table:

```
SELECT * FROM StreetChildrenCareDB.dbo.Orphanages;
```

	OrpID	OrpName	OrpLocation	OrpNumber
1	1	Al-Amin Orphanage	Mirpur, Dhaka	01711-0000...
2	2	Madina Orphanage	Dhanmondi, Dha...	01822-0000...
3	3	Green Valley Orphana...	Gulshan, Dhaka	01933-0000...
4	4	Hope Foundation Home	Uttara, Dhaka	01644-0000...
5	5	Noor Orphanage	Motijheel, Dhaka	01755-0000...

3. Foundations Table:

```
SELECT * FROM StreetChildrenCareDB.dboFOUNDATIONS;
```

	FouID	FouName	SupportType	FouNumber
1	1	BRAC Bangladesh	Education	01711-111001
2	2	Grameen Kalyan	Food	01822-111002
3	3	Dhaka Ahsania Mission	Medical	01933-111003
4	4	An-noor Foundation	Education	01644-111004
5	5	Bidyanondo Foundation	Food	01755-111005

4. Children Table:

```
SELECT * FROM StreetChildrenCareDB.dbo.Children;
```

	ChildID	ChildName	Age	Gender	FoundLocation	ChildStatus	OrpID	FoulD
1	1	Rahim Mia	8	Male	Motijheel, Dhaka	In Orphanage	1	1
2	2	Fatema Beg...	10	Fem...	Sadarghat, Dhaka	In Orphanage	1	2
3	3	Karim Uddin	12	Male	Mirpur, Dhaka	In Orphanage	2	3
4	4	Rima Akter	7	Fem...	Mohammadpur, Dha...	Registered	3	4
5	5	Sajib Hossain	9	Male	Banani, Dhaka	Adopted	4	5

5. Vaccinations Table:

```
SELECT * FROM StreetChildrenCareDB.dbo.Vaccinations;
```

	VacID	ChildID	VacName	GivenDate	NextDueDate	VacStatus
1	1	1	BCG	2024-01-10	2025-01-10	Completed
2	2	1	Polio	2024-03-15	2025-03-15	Completed
3	3	2	BCG	2024-02-20	2025-02-20	Completed
4	4	2	DPT	2024-05-10	2024-11-10	Overdue
5	5	3	Measles	2024-06-01	2025-06-01	Completed

6. Children Full Details (JOIN Query):

```
SELECT
    C.ChildID,
    C.ChildName,
    C.Age,
    C.Gender,
    C.FoundLocation,
    C.ChildStatus,
    O.OrpName,
    F.FouName,
    F.SupportType
FROM Children C
LEFT JOIN Orphanages O ON C.OrpID = O.OrpID
```

LEFT JOIN Foundations F ON C.FouID = F.FouID;

	ChildID	ChildName	Age	Gender	FoundLocation	ChildStatus	OrpName	FouName	SupportType
1	1	Rahim Mia	8	Male	Motijheel, Dhaka	In Orphana...	Al-Amin Orphanage	BRAC Bangladesh	Education
2	2	Fatema Begum	10	Fem...	Sadarghat, Dhaka	In Orphana...	Al-Amin Orphanage	Grameeen Kalyan	Food
3	3	Karim Uddin	12	Male	Mirpur, Dhaka	In Orphana...	Madina Orphanage	Dhaka Ahsania Mission	Medical
4	4	Rima Akter	7	Fem...	Mohammadpur, Dhaka	Registered	Green Valley Orphana...	An-noor Foundation	Education
5	5	Sajib Hossain	9	Male	Banani, Dhaka	Adopted	Hope Foundation Home	Bidyanondo Foundati...	Food

USER STORY

Story ID	Use Case Name	Actor	User Story
US-01	Login	Admin, Staff	A user enters their User ID and Password to log in. The system automatically detects the role and redirects to the correct dashboard.
US-02	View Admin Dashboard	Admin	Admin can view the total count of registered Children, Orphanages, Foundations, and Overdue Vaccinations on the dashboard after login.
US-03	Add New Child Record	Admin	Admin can add a new street child's profile by entering the child's name, age, gender, found location, current status, assigned orphanage, and assigned foundation.
US-04	Update Child Record	Admin	Admin can select any existing child record and update its information when a change is needed.
US-05	Delete Child Record	Admin	Admin can permanently delete a child record from the system when it is no longer needed.
US-06	Search and Sort Children	Admin	Admin can search children by name or status, and sort the list by name, age, or status to find specific records quickly.
US-07	View Child Profile	Admin	Admin can open a detailed profile page for any selected child, which also shows that child's complete vaccination history.
US-08	Add Orphanage	Admin	Admin can register a new orphanage by entering its name, location, and contact number so it becomes available for assigning children.
US-09	Update Orphanage	Admin	Admin can edit the details of any registered orphanage to keep location and contact information up to date.
US-10	Delete Orphanage	Admin	Admin can remove an orphanage record from the system when it is no longer active.
US-11	Add Foundation	Admin	Admin can add a new supporting foundation with its name, support type (Food, Clothing, Education, or Medical), and contact number so it can be linked to children.

US-12	Update and Delete Foundation	Admin	Admin can update or delete any foundation record to keep the list of active support organizations accurate.
US-13	Add Vaccination Record	Admin	Admin can record a vaccination entry for a specific child by selecting the child, vaccine name, given date, next due date, and current status.
US-14	Update Vaccination Status	Admin	Admin can change the vaccination status (Pending, Completed, or Overdue) for any child to keep health records accurate.
US-15	Delete Vaccination Record	Admin	Admin can delete a vaccination record to remove incorrect or duplicate entries from the system.
US-16	Manage Users	Admin	Only Admin can create new user accounts (Admin or Staff), search users by ID or role, and delete non-admin accounts to control system access.
US-17	Logout	Admin	Admin can securely log out of the system at any time.
US-18	Login	Staff	A Staff member can log in with their User ID and Password to access the Staff dashboard.
US-19	View Staff Dashboard	Staff	A Staff member can see the total count of Children, Orphanages, and Foundations on their dashboard after login.
US-20	View Children Records	Staff	A Staff member can view and search the full list of children in read-only mode without any edit or delete access.
US-21	View Child Profile	Staff	A Staff member can view a selected child's complete profile and vaccination history in read-only mode.
US-22	View Orphanage Records	Staff	A Staff member can view the complete list of registered orphanages in read-only mode.
US-23	View Foundation Records	Staff	A Staff member can view all foundation records and their support types in read-only mode.
US-24	View Vaccination Records	Staff	A Staff member can view vaccination records for all children in read-only mode to monitor health status.

US-25	Logout	Staff	A Staff member can securely log out of the system at any time.
-------	--------	-------	--

SYSTEM MODULES

Module	Source File	Responsibility
Login	frmLogin.cs	Handles secure role-based authentication using SQL parameterized queries. Redirects Admin to Admin Dashboard and Staff to Staff Dashboard after successful login.
Admin Dashboard	frmAdminDashboard.cs	Displays real-time summary counts of Children, Orphanages, Foundations, and Overdue Vaccinations. Provides navigation buttons to all management modules.
Staff Dashboard	frmStaffDashboard.cs	Displays the same summary counts as Admin Dashboard but provides only view-only navigation options. Includes a notice that Staff can view and search only.
Children Management	frmChildren.cs	Allows Admin to perform full CRUD operations (Add, Update, Delete, Search, Sort) on child records. Staff can only view and search in read-only mode.
Child Profile	frmChildProfile.cs	Displays the complete profile of a selected child including all personal details and linked vaccination history. Admin can edit or delete the child from this view.
Orphanage Management	frmOrphanage.cs	Allows Admin to add, update, delete, search, and sort orphanage records. Staff can view and search only.
Foundation Management	frmFoundation.cs	Allows Admin to manage supporting foundation records including name, support type, and contact number. Staff can view and search only.
Vaccination Management	frmVaccination.cs	Allows Admin to add, update, delete, search, and sort vaccination records for all children. Staff can view and search only.

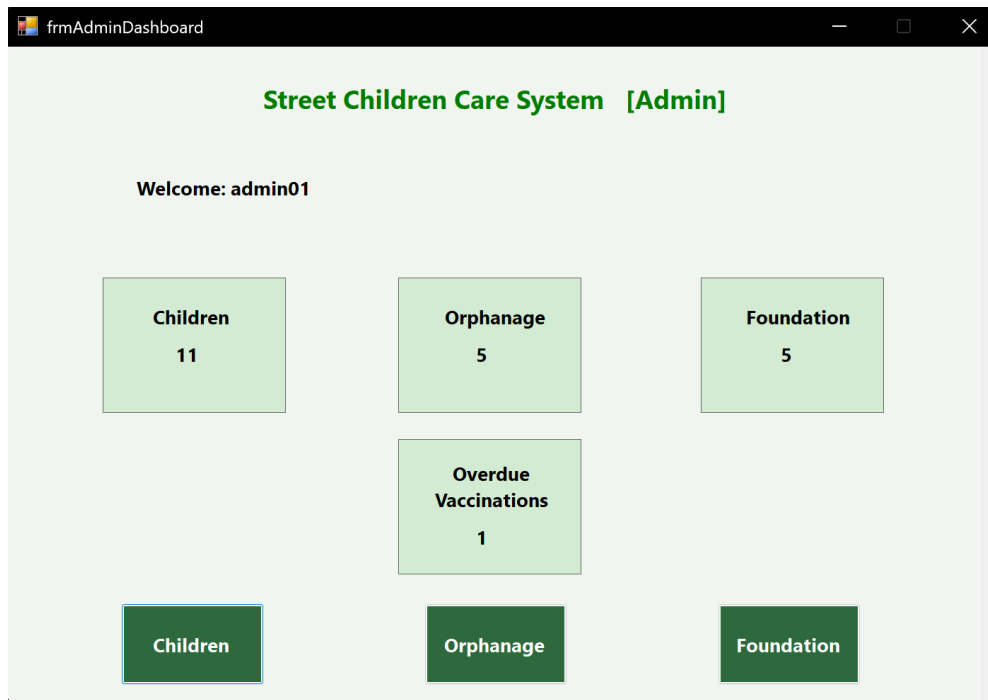
User Management	frmManageUsers.cs	Admin-only module. Allows Admin to create new system user accounts, assign roles (Admin or Staff), search and sort users, and delete accounts.
-----------------	-------------------	--

SCREENSHOTS OF THE APPLICATION

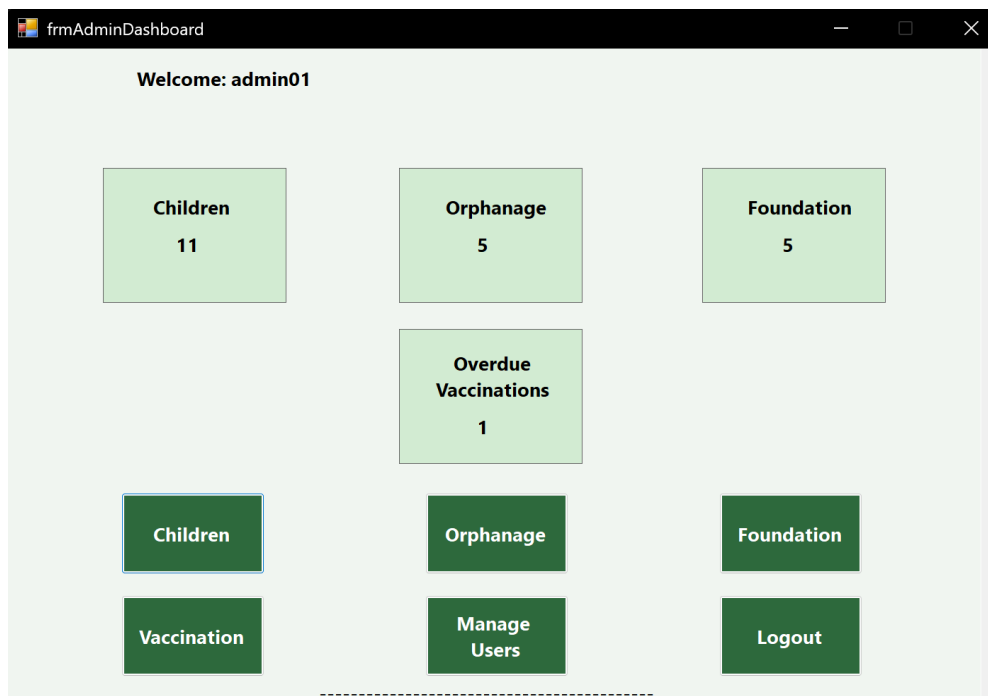
1. Login Page: User logs in with User ID and Password to access the system based on role.

The screenshot shows a Windows application window titled "Login". The window has a dark green header bar with the text "Street Children Care System". Below the header, there is a light green rectangular area containing the login form. The form is titled "Login" and includes two input fields: "User ID :" and "Password :". Below these fields, a message states "System will automatically detect Admin or Staff access after login." At the bottom of the form, there are two buttons: "Login" (dark green) and "Exit" (light green).

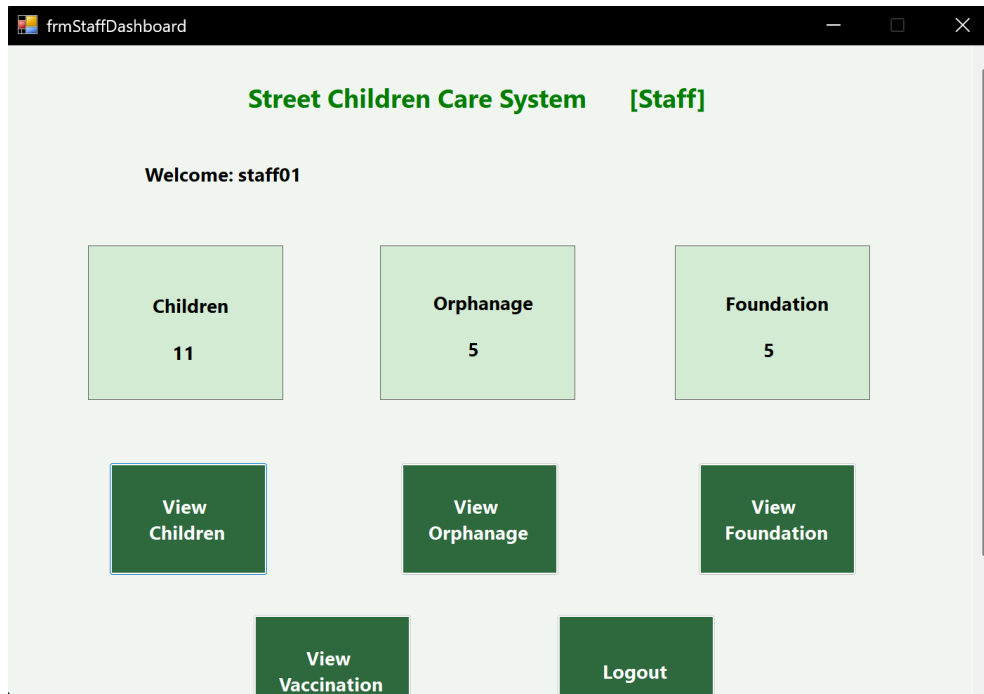
2. Admin Dashboard (Top View): Admin dashboard shows total Children, Orphanages, Foundations, and Overdue Vaccinations with navigation buttons.



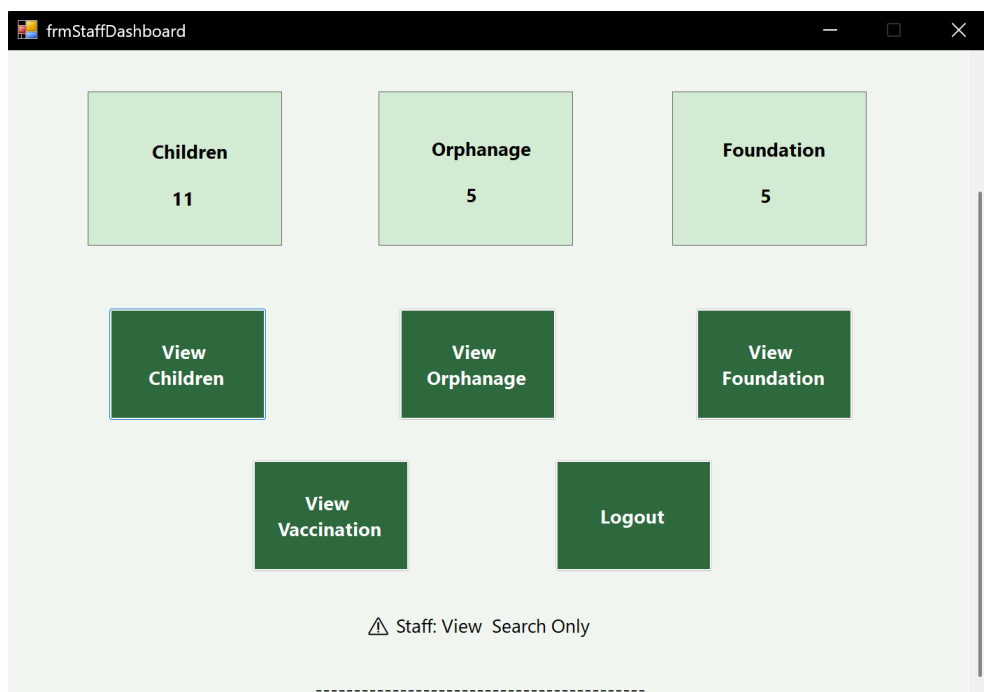
3. Admin Dashboard (Bottom View): Admin dashboard provides Vaccination, Manage Users, and Logout options for full system control.



4. Staff Dashboard (Top View): Staff dashboard shows summary information with options to view different modules.



5. Staff Dashboard (Bottom View): Staff can only view and search records without edit or delete access.



6. Children Management Form (Top View): Admin can add, update, and clear child information using this form.

The screenshot shows a web application window titled 'frmChildren'. The main heading is 'Children Management'. There are two buttons at the top right: 'Home' and 'Logout'. The form contains the following fields:

- Child Name :** Text input with value 'Rahim Mia'
- Age :** Spin box with value '8'
- Gender :** Dropdown menu with value 'Male'
- Location :** Text input with value 'Motijheel, Dhaka'
- Status :** Dropdown menu with value 'In Orphanage'
- Orphanage :** Dropdown menu with value 'Al-Amin Orphanage'
- Foundation :** Dropdown menu with value 'BRAC Bangladesh'

At the bottom, there are three buttons: 'Save', 'Update', and 'Clear'.

7. Children Management Form (Bottom View): All child records are shown with search, sort, profile view, and delete options.

The screenshot shows the same web application window. At the top, there is a search and sort section with a 'Search' button. Below this is a table of child records. The table has the following columns: ChildID, ChildName, Age, Gender, FoundLoca, ChildStatus, OrpName, and FouName. The table contains 14 rows of data. At the bottom, there are two buttons: 'View Profile' and 'Delete'.

ChildID	ChildName	Age	Gender	FoundLoca	ChildStatus	OrpName	FouName
1	Rahim ...	8	Male	Motijhe...	In Orpha...	Al-Amin ...	BRAC Ba...
2	Fatema ...	10	Female	Sadargh...	In Orpha...	Al-Amin ...	Gramee...
3	Karim U...	12	Male	Mirpur, ...	In Orpha...	Madina ...	Dhaka A...
4	Rima Ak...	7	Female	Moham...	Register...	Green V...	An-noor...
5	Sajib Ho...	9	Male	Banani, ...	Adopted	Hope Fo...	Bidyano...
6	hashim	9	Male	Mirpur	In Orpha...	Al-Amin ...	BRAC Ba...
8	Akib	7	Male	Kuril	Register...	Al-Amin ...	BRAC Ba...
9	Mina	5	Female	Mohakh...	Register...		
11	Raju	10	Male	Muqhdha	In Orpha...	Madina ...	An-noor...
13	Runa	10	Female	Mirpur 11	Adopted	Hope Fo...	Bidyano...
14	Test 2	7	Male	ABC	Adopted	Green V...	BRAC Ba...

8. Child Profile Form (Top View): Child profile displays complete information of the selected child.

The screenshot shows a web application window titled 'frmChildProfile'. The main heading is 'Child Profile'. There are two buttons at the top right: 'Home' and 'Logout'. The child's information is displayed in a central box:

- Child ID : 1
- Name : Rahim Mia
- Age : 8 Gender : Male
- Found At : Motijheel, Dhaka
- Status : In Orphanage
- Orphanage : Al-Amin Orphanage
- Foundation : BRAC Bangladesh

At the bottom of the form, there are two buttons: 'Edit Profile' and 'Delete Child'.

9. Child Profile Form (Bottom View): Vaccination records of the selected child are displayed with the add option.

The screenshot shows the same web application window, but now displaying vaccination records. The 'Edit Profile' and 'Delete Child' buttons are at the top. Below them is a section titled '— Vaccination Records —' with a 'Sort :' dropdown menu. A table displays the vaccination records:

	VacID	VacName	GivenDate	NextDueDate	VacStatus
*	2	Polio	3/15/2024	3/15/2025	Completed
*	1	BCG	1/10/2024	1/10/2025	Completed

Below the table, there is a large grey rectangular area. At the bottom of the form, there are two buttons: 'Add Vaccination' and 'Back'.

10. Orphanage Management Form (Top View): Admin can add, update, and manage orphanage information from this form.

The screenshot shows a web application window titled 'frmOrphanage'. The main heading is 'Orphanage Management'. There are two buttons at the top right: 'Home' and 'Logout'. Below the heading, there are three input fields: 'Orphanage Name : Al-Amin Orphanage', 'Location : Mirpur, Dhaka', and 'Orphanage Contact No. : 01711-000001'. Below these fields are three buttons: 'Save', 'Update', and 'Clear'. Further down, there is a 'Search : ' input field and a 'Sort : ' dropdown menu set to 'Sort by Name'. A 'Search' button is next to the search input. At the bottom, there is a table with the following data:

OrpID	OrpName	OrpLocation	OrpNumber
1	Al-Amin Orphanage	Mirpur, Dhaka	01711-000001
2	Green Valley Orpha...	Gulshan, Dhaka	01933-000003

11. Orphanage Management Form (Bottom View): All orphanage records are displayed with delete and back options.

The screenshot shows the same web application window, but in the 'Bottom View'. The 'Save', 'Update', and 'Clear' buttons are at the top. Below them are the 'Search : ' input field and the 'Sort : ' dropdown menu set to 'Sort by Name', with a 'Search' button. The table below shows a list of orphanage records:

OrpID	OrpName	OrpLocation	OrpNumber
1	Al-Amin Orphanage	Mirpur, Dhaka	01711-000001
3	Green Valley Orpha...	Gulshan, Dhaka	01933-000003
4	Hope Foundation H...	Uttara, Dhaka	01644-000004
2	Madina Orphanage	Dhanmondi, Dhaka	01822-000002
5	Noor Orphanage	Motijheel, Dhaka	01755-000005

Below the table, there are two buttons: 'Delete' (in red) and 'Back' (in green).

12. Foundation Management Form (Top View): Admin can manage foundation details with save, update, and clear options.

The screenshot shows a web application window titled 'frmFoundation'. It features a 'Foundation Management' header with 'Home' and 'Logout' links. The form includes three input fields: 'Foundation Name' (containing 'An-noor Foundation'), 'Support Type' (a dropdown menu set to 'Education'), and 'Foundation Number' (containing '01644-111004'). Below these fields are three buttons: 'Save', 'Update', and 'Clear'. A search section contains a 'Search' input field and a 'Search' button. A 'Sort' dropdown menu is set to 'Sort by Name'. At the bottom, a table displays foundation records.

	FouID	FouName	SupportType	FouNumber
▶	4	An-noor Foundati...	Education	01644-111004
	5	Bidyanondo Foun...	Food	01755-111005
	1	BRAC Bangladesh	Education	01711-111001

13. Foundation Management Form (Bottom View): All foundation records are shown with delete and back options.

The screenshot shows the same web application window, but in the 'Bottom View'. The 'Save', 'Update', and 'Clear' buttons are at the top. The search and sort section remains. The table now displays five records. Below the table is a large grey rectangular area, and at the bottom are 'Delete' and 'Back' buttons.

	FouID	FouName	SupportType	FouNumber
▶	4	An-noor Foundati...	Education	01644-111004
	5	Bidyanondo Foun...	Food	01755-111005
	1	BRAC Bangladesh	Education	01711-111001
	3	Dhaka Ahsania Mi...	Medical	01933-111003
	2	Grameeen Kalyan	Food	01822-111002

14. Vaccination Management Form (Top View): Admin can add and manage vaccination information for children.

The screenshot shows the 'frmVaccination' window with the title 'Vaccination Management'. It features a header with 'Home' and 'Logout' buttons. The main form area contains several input fields: 'Child' (Rahim Mia), 'Vaccine Name' (BCG), 'Given Date' (Tuesday, October 1, 2024), 'Next Due Date' (Wednesday, October 1, 2025), and 'Vaccine Status' (Completed). Below these fields are 'Save', 'Update', and 'Clear' buttons. A section titled 'All Vaccination Records' includes a 'Search' input field and a 'Sort' dropdown menu set to 'Sort by Status', with a 'Search' button next to it.

15. Vaccination Management Form (Bottom View): All vaccination records are displayed with search and sort features.

The screenshot shows the 'frmVaccination' window with the title 'Vaccination Management'. It features a header with 'Save', 'Update', and 'Clear' buttons. The main form area contains a section titled 'All Vaccination Records' with a 'Search' input field and a 'Sort' dropdown menu set to 'Sort by Status', with a 'Search' button next to it. Below this is a table displaying vaccination records:

VacID	ChildName	VacName	GivenDate	NextDueDate	VacStatus
1	Rahim Mia	BCG	1/10/2024	1/10/2025	Completed
2	Rahim Mia	Polio	3/15/2024	3/15/2025	Completed
3	Fatema Bequ...	BCG	2/20/2024	2/20/2025	Completed
5	Karim Uddin	Measles	6/1/2024	6/1/2025	Completed
7	Rima Akter	BCG	9/20/2024	9/20/2025	Completed
10	Test 2	BCG	5/16/2026	5/16/2026	Completed
4	Fatema Bequ...	DPT	5/10/2024	11/10/2024	Overdue
8	Sajib Hossain	Hepatitis B	10/5/2024	4/5/2025	Pending
6	Karim Uddin	Polio	8/15/2024	2/15/2025	Pending

Below the table are 'Delete' and 'Back' buttons.

16. Manage Users Form (Top View): Admin can create new user accounts and manage user roles.

The screenshot shows the 'frmManageUsers' window. At the top, it says 'Manage Users (Admin Only)' with 'Home' and 'Logout' buttons. Below this are input fields for 'UserID' (admin01), 'Password' (admin123), and 'UserRole' (Admin). There are 'Save' and 'Clear' buttons. Further down are 'Search' and 'Sort' (Sort by Role) sections. At the bottom is a table of users.

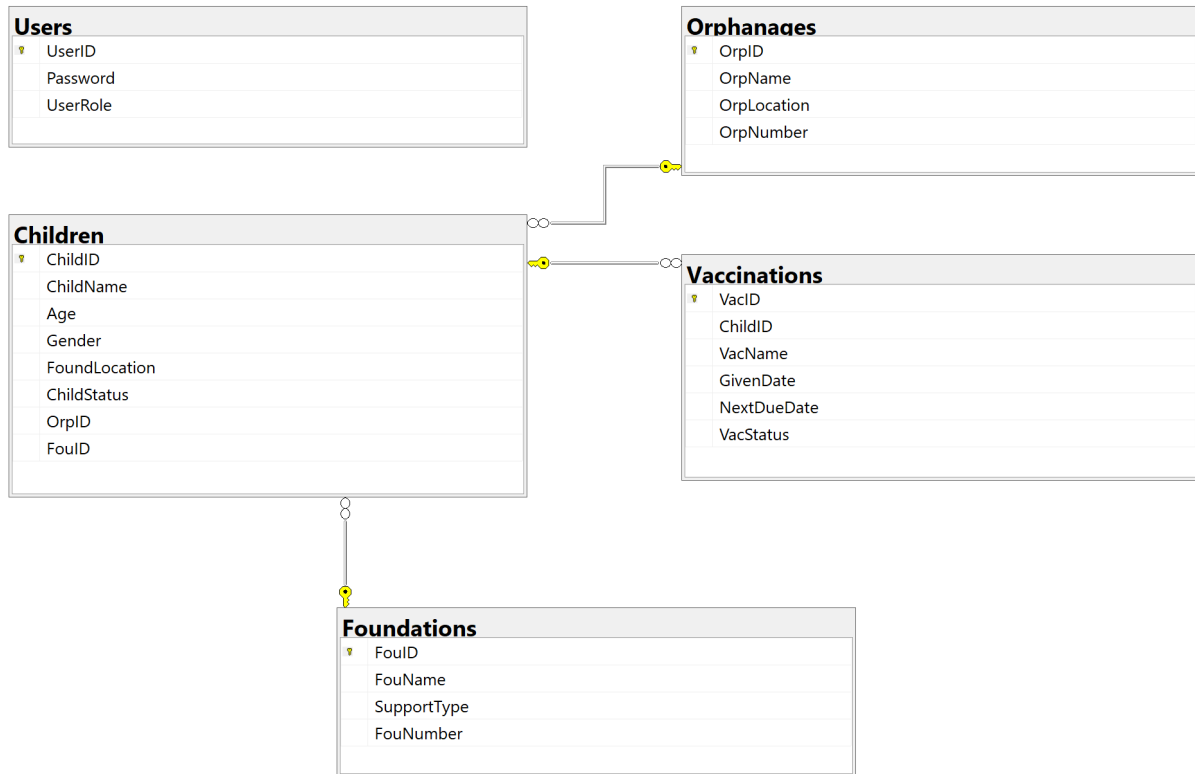
	UserID	Password	UserRole
▶	admin01	admin123	Admin
	admin02	admin456	Admin
	staff01	staff123	Staff
	staff02	staff456	Staff

17. Manage Users Form (Bottom View): All system users are displayed with delete and back options.

The screenshot shows the 'frmManageUsers' window in a different state. The 'UserRole' dropdown is set to 'Admin'. There are 'Save' and 'Clear' buttons. Below are 'Search' and 'Sort' (Sort by Role) sections. The table now lists more users, including staff03 and staff04. At the bottom, there are 'Delete' and 'Back' buttons.

	UserID	Password	UserRole
▶	admin01	admin123	Admin
	admin02	admin456	Admin
	staff01	staff123	Staff
	staff02	staff456	Staff
	staff03	staff789	Staff
	staff04	staff101112	Staff

SCHEMA DIAGRAM



IMPLEMENTATION

1. Login Module:

File Name: [frmLogin.cs](#)

```
using System;
using System.Data;
using System.Windows.Forms;
using System.Data.SqlClient;
using StreetChildrenCareSystem.Forms;

namespace StreetChildrenCareSystem
{
    public partial class frmLogin : Form
    {
        public frmLogin()
        {
            InitializeComponent();

            // Handles user login validation and
            // access control
            private void btnLogin_Click(object
            sender, EventArgs e)
            {
                // Validate if User ID is provided
                if
                (string.IsNullOrEmpty(txtUserID.T
                ext))
                {
                    MessageBox.Show("Please
                    enter User ID.", "Validation",
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Warning);
                    txtUserID.Focus();
                    return;
                }

                // Validate if Password is
                // provided
                if
                (string.IsNullOrEmpty(txtPassword
                .Text))
                {
                    MessageBox.Show("Please
                    enter Password.", "Validation",
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Warning);
                    txtPassword.Focus();
                    return;
                }

                try
                {
                    // Check database for matching
                    // user credentials
                    string query = "SELECT
                    UserID, UserRole FROM Users
                    WHERE UserID = @uid AND Password
                    = @pass";
                    SqlParameter[] parameters = {
                        new SqlParameter("@uid",
                        txtUserID.Text.Trim()),
                        new SqlParameter("@pass",
                        txtPassword.Text.Trim())
                    };

                    DataTable dt =
                    DBHelper.GetData(query, parameters);
                }
            }
        }
    }
}
```

```

        // If user is found, redirect
        based on their role
        if (dt.Rows.Count > 0)
        {
            string role =
dt.Rows[0]["UserRole"].ToString();

            // Route to Admin
Dashboard
            if (role == "Admin")
            {
                frmAdminDashboard
adminForm = new
frmAdminDashboard(txtUserID.Text);
                adminForm.Show();
                this.Hide();
            }
            // Route to Staff Dashboard
            else if (role == "Staff")
            {
                frmStaffDashboard
staffForm = new
frmStaffDashboard(txtUserID.Text);
                staffForm.Show();
                this.Hide();
            }
        }
        else

```

```

        {
            MessageBox.Show("Invalid
User ID or Password.", "Login Failed",
MessageBoxButtons.OK,
MessageBoxIcon.Error);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error: " +
ex.Message);
    }
}

// Closes the entire application
private void btnExit_Click(object
sender, EventArgs e)
{
    Application.Exit();
}

private void frmLogin_Load(object
sender, EventArgs e)
{
}
}
}

```

2. Add/Register User Module (Admin Only):

File Name: [frmManageUsers.cs](#)

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace
StreetChildrenCareSystem.Forms
{
    public partial class frmManageUsers :
    Form
    {
        private string selectedUserID = "";

        // Constructor: only Admin can
open this form
        public frmManageUsers()
        {
            InitializeComponent();

            // Setup drop-down options and list
users on startup
            private void
frmManageUsers_Load(object sender,
EventArgs e)
            {
                // Role options for new users

                cmbUserRole.Items.Add("Admin");

                cmbUserRole.Items.Add("Staff");
                cmbUserRole.SelectedIndex = 0;

                // Sort options
                cmbSort.Items.Add("Sort by
Role");
                cmbSort.Items.Add("Sort by
UserID");
                cmbSort.SelectedIndex = 0;

                LoadUsers();

                dgvUsers.AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode.Fi
ll;
            }

            // Load all users from database
            private void LoadUsers()
            {
                DataTable dt =
DBHelper.GetData(
                "SELECT UserID, Password,
UserRole" +
                " FROM Users ORDER BY
UserRole ASC");
                dgvUsers.DataSource = dt;
            }

            // Save new user
```

```

        private void btnSave_Click(object sender, EventArgs e)
        {
            if
            (string.IsNullOrEmpty(txtUserID.Text))
            {
                MessageBox.Show("User ID is required!");
                txtUserID.Focus();
                return;
            }

            if
            (string.IsNullOrEmpty(txtPassword.Text))
            {
                MessageBox.Show("Password is required!");
                txtPassword.Focus();
                return;
            }

            // Check if UserID already exists
            DataTable check =
            DBHelper.GetData(
                "SELECT UserID FROM
                Users WHERE UserID = @uid",
                new SqlParameter[] { new
                SqlParameter("@uid",
                txtUserID.Text.Trim()) });

            if (check.Rows.Count > 0)
            {
                MessageBox.Show("This User
                ID already exists! Choose a different
                one.");
            }
        }
    }

```

```

        txtUserID.Focus();
        return;
    }

    // Insert new account details into
    database
    string query =
        "INSERT INTO Users
        (UserID, Password, UserRole)" +
        " VALUES (@uid, @pass,
        @role)";

    SqlParameter[] p =
    {
        new SqlParameter("@uid",
        txtUserID.Text.Trim()),
        new SqlParameter("@pass",
        txtPassword.Text.Trim()),
        new SqlParameter("@role",
        cmbUserRole.Text)
    };

    if
    (DBHelper.ExecuteQuery(query, p))
    {
        MessageBox.Show("User
        Added Successfully ");
        ClearForm();
        LoadUsers();
    }
}

// Clicking a row fills the input
fields for updating/deleting
private void
dgvUsers_CellClick(object sender,
DataGridViewCellEventArgs e)

```

```

    {
        if (e.RowIndex >= 0)
        {
            DataGridViewRow row =
dgvUsers.Rows[e.RowIndex];
            selectedUserID =
row.Cells["UserID"].Value.ToString();
            txtUserID.Text =
selectedUserID;
            txtPassword.Text =
row.Cells["Password"].Value.ToString();
            cmbUserRole.Text =
row.Cells["UserRole"].Value.ToString();
        }
    }

    // Delete selected user
    private void btnDelete_Click(object
sender, EventArgs e)
    {
        if
(string.IsNullOrEmpty(selectedUserID))
        {
            MessageBox.Show("Please
select a User first!");
            return;
        }

        // Protect the main admin account
from being deleted
        if (selectedUserID.ToLower() ==
"admin01" ||
            selectedUserID.ToLower() ==
"admin")
        {
            MessageBox.Show(

```

```

            "Cannot delete the main
Admin account!",
            "Not Allowed",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
            return;
        }

        // Ask for confirmation before
deleting
        DialogResult dr =
MessageBox.Show(
            "Delete User: " +
selectedUserID + "?", "Confirm",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Warning);

        if (dr == DialogResult.Yes)
        {
            if (DBHelper.ExecuteQuery(
                "DELETE FROM Users
WHERE UserID = @uid",
                new SqlParameter[] { new
SqlParameter("@uid", selectedUserID)
}))
            {
                MessageBox.Show("User
Deleted ");
                ClearForm();
                LoadUsers();
            }
        }
    }

    // Search users by matching UserID
or Role keywords

```

```

        private void btnSearch_Click(object
sender, EventArgs e)
        {
            string kw =
txtSearch.Text.Trim();

            // If search box is empty, reload
all users
            if (string.IsNullOrEmpty(kw))
            {
                LoadUsers();
                return;
            }

            DataTable dt =
DBHelper.GetData(
                "SELECT UserID, Password,
UserRole FROM Users" +
                " WHERE UserID LIKE @kw
OR UserRole LIKE @kw",
                new SqlParameter[] { new
SqlParameter("@kw", "%" + kw + "%")
});

            dgvUsers.DataSource = dt;

            if (dt.Rows.Count == 0)
                MessageBox.Show("No users
found matching: " + kw);
        }

        // Rearrange data table grid sorting
sequence
        private void
cmbSort_SelectedIndexChanged(object
sender, EventArgs e)
        {

```

```

            // Ignore if nothing is selected yet
            if
(string.IsNullOrEmpty(cmbSort.Text))
                return;

            string sortBy = "";

            if (cmbSort.Text == "Sort by
Role")
                sortBy = "UserRole ASC";
            else if (cmbSort.Text == "Sort by
UserID")
                sortBy = "UserID ASC";

            if (sortBy == "") return;

            DataTable dt =
DBHelper.GetData(
                "SELECT UserID, Password,
UserRole" +
                " FROM Users ORDER BY "
+ sortBy);

            dgvUsers.DataSource = dt;
        }

        // Clear all input fields
        private void ClearForm()
        {
            txtUserID.Clear();
            txtPassword.Clear();
            txtSearch.Clear();
            cmbUserRole.SelectedIndex = 0;
            selectedUserID = "";
        }

        // Clear button trigger logic

```

```

        private void btnClear_Click(object
sender, EventArgs e)
        {
            ClearForm();
            LoadUsers();
        }

        // Home: show the hidden
        dashboard and close this form
        private void btnHome_Click(object
sender, EventArgs e)
        {
            foreach (Form openForm in
Application.OpenForms)
            {
                if (openForm is
frmAdminDashboard)
                {
                    openForm.Show();
                    break;
                }
            }
            this.Close();
        }

        // Back: same as Home for this form
        private void btnBack_Click(object
sender, EventArgs e)
        {
            foreach (Form openForm in
Application.OpenForms)
            {
                if (openForm is
frmAdminDashboard)
                {
                    openForm.Show();
                    break;

```

```

        }
    }
    this.Close();
}

// Process profile logouts and safe
tracking resets
private void
btnLogout_Click(object sender,
EventArgs e)
{
    DialogResult result =
MessageBox.Show(
        "Are you sure to logout?",
        "Logout",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

    if (result == DialogResult.Yes)
    {
        Form dashboardForm = null;
        foreach (Form openForm in
Application.OpenForms)
        {
            if (openForm is
frmAdminDashboard || openForm is
frmStaffDashboard)
                dashboardForm =
openForm;
        }
        if (dashboardForm != null)
            dashboardForm.Close();

        frmLogin login = new
frmLogin();
        login.Show();
        this.Close();

```

```

    }
}
    }
}

```

3. Admin Dashboard Module:

File Name: [frmAdminDashboard.cs](#)

```

using System;
using System.Data;
using System.Windows.Forms;

namespace
StreetChildrenCareSystem.Forms
{
    public partial class
    frmAdminDashboard : Form
    {
        private string loggedUserID;

        // Initialize dashboard state with
        login credentials
        public frmAdminDashboard(string
        userID)
        {
            InitializeComponent();
            loggedUserID = userID;
            lblWelcome.Text = "Welcome: "
+ userID;
        }

        private void
        frmAdminDashboard_Load(object
        sender, EventArgs e)
        {
            LoadDashboardCounts();
        }
    }
}

```

```

// Fetches total metric records from
the system tables
private void
LoadDashboardCounts()
{
    try
    {
        // Count registered children
        DataTable dtC =
        DBHelper.GetData("SELECT
COUNT(*) AS Total FROM Children");
        lblChildCount.Text =
        dtC.Rows[0]["Total"].ToString();

        // Count registered orphanages
        DataTable dtO =
        DBHelper.GetData("SELECT
COUNT(*) AS Total FROM
Orphanages");
        lblOrpCount.Text =
        dtO.Rows[0]["Total"].ToString();

        // Count registered charity
        foundations
        DataTable dtF =
        DBHelper.GetData("SELECT
COUNT(*) AS Total FROM
Foundations");
        lblFouCount.Text =
        dtF.Rows[0]["Total"].ToString();
    }
}

```

```

        // Count vaccination statuses
        tagged as overdue
        DataTable dtV =
        DBHelper.GetData("SELECT
        COUNT(*) AS Total FROM
        Vaccinations WHERE VacStatus =
        'Overdue'");
        lblOverdueCount.Text =
        dtV.Rows[0]["Total"].ToString();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error: " +
        ex.Message);
    }
}

private void
btnChildren_Click(object sender,
EventArgs e)
{
    // Open Children form and hide
    the dashboard (don't close it)
    frmChildren f = new
    frmChildren("Admin", loggedUserID);
    f.Show();
    this.Hide(); // Hide dashboard so
    it can be shown again when Home is
    clicked
}

private void
btnOrphanage_Click(object sender,
EventArgs e)
{

```

```

        // Open Orphanage form and hide
        dashboard
        frmOrphanage f = new
        frmOrphanage("Admin", loggedUserID);
        f.Show();
        this.Hide();
    }

    private void
    btnFoundation_Click(object sender,
    EventArgs e)
    {
        // Open Foundation form and
        hide dashboard
        frmFoundation f = new
        frmFoundation("Admin",
        loggedUserID);
        f.Show();
        this.Hide();
    }

    private void
    btnVaccination_Click(object sender,
    EventArgs e)
    {
        // Open Vaccination form and
        hide dashboard
        frmVaccination f = new
        frmVaccination("Admin");
        f.Show();
        this.Hide();
    }

    private void
    btnManageUsers_Click(object sender,
    EventArgs e)
    {

```

```

        // Open Manage Users form and
hide dashboard (Admin only)
        frmManageUsers f = new
frmManageUsers();
        f.Show();
        this.Hide();
    }

    // Confirms sign out action and
returns window tracking to login form
    private void
btnLogout_Click(object sender,
EventArgs e)
    {
        DialogResult result =
MessageBox.Show(
            "Are you sure to logout?",
"Logout",

```

```

        MessageBoxButtons.YesNo,
MessageBoxIcon.Question);

        if (result == DialogResult.Yes)
        {
            frmLogin login = new
frmLogin();
            login.Show();
            this.Close();
        }
    }

    private void label4_Click(object
sender, EventArgs e)
    {
    }
}

```

4. Children Management Module:

File Name: [frmChildren.cs](#)

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace
StreetChildrenCareSystem.Forms
{
    public partial class frmChildren :
    Form
    {
        // Global variables for user session
        and selection tracking
        private string userRole;
        private string userID;
        private int selectedChildID = 0;

        // Constructor: receives user details
        from login or dashboard
        public frmChildren(string role,
        string uid)
        {
            InitializeComponent();
            userRole = role;
            userID = uid;
        }

        // Setup form permissions,
        dropdowns, and grid data on load
        private void
        frmChildren_Load(object sender,
        EventArgs e)
        {
            SetPermissions();

            LoadOrphanageCombo();
            LoadFoundationCombo();
            LoadChildren();

            dgvChildren.AutoSizeColumnsMode =
            DataGridViewAutoSizeColumnsMode.Fi
            ll;
        }

        // Role-Based Access Control:
        restricts Staff to read-only access
        private void SetPermissions()
        {
            if (userRole == "Staff")
            {
                btnSave.Visible = false;
                btnUpdate.Visible = false;
                btnDelete.Visible = false;
                btnClear.Visible = false;
                txtChildName.ReadOnly =
                true;

                numAge.Enabled = false;
                cmbGender.Enabled = false;
                txtLocation.ReadOnly = true;
                cmbStatus.Enabled = false;
                cmbOrphanage.Enabled =
                false;

                cmbFoundation.Enabled =
                false;
            }
        }
    }
}
```

```

        // Load orphanages into ComboBox
        and add a default "None" option
        private void
        LoadOrphanageCombo()
        {
            // 1. Fetch the data from the
            database
            DataTable dt =
            DBHelper.GetData("SELECT OrpID,
            OrpName FROM Orphanages");

            // 2. Create a new dummy row
            with the same structure as the table
            DataRow dr = dt.NewRow();
            dr["OrpID"] = 0;           // Use 0
            as the ID for 'None'
            dr["OrpName"] = "-- None --"; //
            The text the user will see

            // 3. Insert this row at the very top
            (index 0)
            dt.Rows.InsertAt(dr, 0);

            // 4. Bind to the ComboBox
            cmbOrphanage.DisplayMember
            = "OrpName";
            cmbOrphanage.ValueMember =
            "OrpID";
            cmbOrphanage.DataSource = dt;
        }

        // Load foundations into ComboBox
        and add a default "None" option
        private void
        LoadFoundationCombo()
        {

```

```

            DataTable dt =
            DBHelper.GetData("SELECT FouID,
            FouName FROM Foundations");

            DataRow dr = dt.NewRow();
            dr["FouID"] = 0;
            dr["FouName"] = "-- None --";
            dt.Rows.InsertAt(dr, 0);

            cmbFoundation.DisplayMember
            = "FouName";
            cmbFoundation.ValueMember =
            "FouID";
            cmbFoundation.DataSource = dt;
        }

        // Fetch and display all children
        records with connected orphanage and
        foundation names
        private void LoadChildren()
        {
            string query =
            "SELECT C.ChildID,
            C.ChildName, C.Age, C.Gender," +
            " C.FoundLocation,
            C.ChildStatus, O.OrpName, F.FouName"
            +
            " FROM Children C" +
            " LEFT JOIN Orphanages O
            ON C.OrpID = O.OrpID" +
            " LEFT JOIN Foundations F
            ON C.FouID = F.FouID";

            DataTable dt =
            DBHelper.GetData(query);
            dgvChildren.DataSource = dt;
        }

```

```

// Filter children records using
search keywords for Name or Status
private void btnSearch_Click(object
sender, EventArgs e)
{
    string keyword =
txtSearch.Text.Trim();

    string query =
        "SELECT C.ChildID,
C.ChildName, C.Age, C.Gender," +
        " C.ChildStatus, O.OrpName,
F.FouName" +
        " FROM Children C" +
        " LEFT JOIN Orphanages O
ON C.OrpID = O.OrpID" +
        " LEFT JOIN Foundations F
ON C.FouID = F.FouID" +
        " WHERE C.ChildName LIKE
@keyword OR C.ChildStatus LIKE
@keyword";

    SqlParameter[] p = { new
SqlParameter("@keyword", "%" +
keyword + "%") };

    DataTable dt =
DBHelper.GetData(query, p);
    dgvChildren.DataSource = dt;
}

// Re-query and sort children
records based on selected dropdown
sorting option

```

```

private void
cmbSort_SelectedIndexChanged(object
sender, EventArgs e)
{
    string sortBy = "";

    if (cmbSort.Text == "Sort by
Name")
        sortBy = "C.ChildName ASC";
    else if (cmbSort.Text == "Sort by
Status")
        sortBy = "C.ChildStatus ASC";
    else if (cmbSort.Text == "Sort by
Age")
        sortBy = "C.Age ASC";

    if (sortBy == "") return;

    string query =
        "SELECT C.ChildID,
C.ChildName, C.Age, C.Gender," +
        " C.ChildStatus, O.OrpName,
F.FouName" +
        " FROM Children C" +
        " LEFT JOIN Orphanages O
ON C.OrpID = O.OrpID" +
        " LEFT JOIN Foundations F
ON C.FouID = F.FouID" +
        " ORDER BY " + sortBy;

    DataTable dt =
DBHelper.GetData(query);
    dgvChildren.DataSource = dt;
}

// Validate and save a new child
record to the database

```

```

        private void btnSave_Click(object sender, EventArgs e)
        {
            if
            (string.IsNullOrEmpty(txtChildName.Text))
            {
                MessageBox.Show("Child Name required!");
                return;
            }

            string query =
                "INSERT INTO Children
            (ChildName, Age, Gender, " +
                " FoundLocation, ChildStatus,
            OrpID, FouID)" +
                " VALUES (@name, @age,
            @gender, @loc, @status, @orp, @fou)";

            // Convert 0 selection to database
            NULL values
            object orpIdParam =
            (int)cmbOrphanage.SelectedValue == 0 ?
            DBNull.Value :
            cmbOrphanage.SelectedValue;
            object fouIdParam =
            (int)cmbFoundation.SelectedValue == 0
            ? DBNull.Value :
            cmbFoundation.SelectedValue;

            SqlParameter[] p =
            {
                new SqlParameter("@name",
            txtChildName.Text.Trim()),
                new SqlParameter("@age",
            (int)numAge.Value),

```

```

                new SqlParameter("@gender",
            cmbGender.Text),
                new SqlParameter("@loc",
            txtLocation.Text.Trim()),
                new SqlParameter("@status",
            cmbStatus.Text),
                new SqlParameter("@orp",
            orpIdParam),
                new SqlParameter("@fou",
            fouIdParam)
            };

```

```

            bool result =
            DBHelper.ExecuteQuery(query, p);
            if (result)
            {
                MessageBox.Show("Child
            Saved ");
                ClearForm();
                LoadChildren();
            }
        }

```

```

        // Fill form input fields using the
        data from the selected table row
        private void
        dgvChildren_CellClick(object sender,
        DataGridViewCellEventArgs e)
        {
            if (e.RowIndex >= 0)
            {
                DataGridViewRow row =
            dgvChildren.Rows[e.RowIndex];

```

```

            // 1. Handle safe retrieval of
            Child ID

```

```

        if (row.Cells["ChildID"].Value
!= DBNull.Value &&
row.Cells["ChildID"].Value != null)
        {
            selectedChildID =
Convert.ToInt32(row.Cells["ChildID"].V
alue);
        }
        else
        {
            selectedChildID = 0;
        }

        // 2. Set Text Field values
        txtChildName.Text =
Convert.ToString(row.Cells["ChildName
"].Value);

        // 3. Set Numeric values safely
        if (row.Cells["Age"].Value !=
DBNull.Value &&
row.Cells["Age"].Value != null)
        {
            numAge.Value =
Convert.ToDecimal(row.Cells["Age"].Va
lue);
        }
        else
        {
            numAge.Value = 0;
        }

        // 4. Handle Orphanage
        ComboBox selection display
        string orpName =
Convert.ToString(row.Cells["OrpName"]
.Value);

```

```

        cmbOrphanage.Text =
string.IsNullOrEmpty(orpName) ? "--
None --" : orpName;

        // 5. Handle Foundation
        ComboBox selection display
        string fouName =
Convert.ToString(row.Cells["FouName"]
.Value);
        cmbFoundation.Text =
string.IsNullOrEmpty(fouName) ? "--
None --" : fouName;
    }

    // Update information for the
currently selected child record
    private void
btnUpdate_Click(object sender,
EventArgs e)
    {
        if (selectedChildID == 0)
        {
            MessageBox.Show("Please
select a child!");
            return;
        }

        string query =
            "UPDATE Children SET
ChildName = @name, Age = @age," +
            " Gender = @gender,
FoundLocation = @loc," +
            " ChildStatus = @status,
OrpID = @orp, FouID = @fou" +
            " WHERE ChildID = @id";

```

```

        // Convert 0 selection to database
        NULL values
        object orpIdValue =
(int)cmbOrphanage.SelectedValue == 0 ?
DBNull.Value :
cmbOrphanage.SelectedValue;
        object fouIdValue =
(int)cmbFoundation.SelectedValue == 0
? DBNull.Value :
cmbFoundation.SelectedValue;

        SqlParameter[] p =
        {
            new SqlParameter("@name",
txtChildName.Text.Trim()),
            new SqlParameter("@age",
(int)numAge.Value),
            new SqlParameter("@gender",
cmbGender.Text),
            new SqlParameter("@loc",
txtLocation.Text.Trim()),
            new SqlParameter("@status",
cmbStatus.Text),
            new SqlParameter("@orp",
orpIdValue),
            new SqlParameter("@fou",
fouIdValue),
            new SqlParameter("@id",
selectedChildID)
        };

        bool result =
DBHelper.ExecuteQuery(query, p);
        if (result)
        {
            MessageBox.Show("Updated
Successfully ");

```

```

        ClearForm();
        LoadChildren();
    }
}

// Delete a child record along with
their linked vaccination history
private void btnDelete_Click(object
sender, EventArgs e)
{
    if (selectedChildID == 0)
    {
        MessageBox.Show("Please
select a child!");
        return;
    }

    DialogResult dr =
MessageBox.Show(
        "Are you sure to delete?",
        "Confirm Delete",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Warning);

    if (dr == DialogResult.Yes)
    {
        // Delete linked child
        vaccinations first to maintain database
        integrity
        DBHelper.ExecuteQuery(
            "DELETE FROM
Vaccinations WHERE ChildID = @id",
            new SqlParameter[] { new
SqlParameter("@id", selectedChildID)
});

        // Delete the main child record

```

```

        bool result =
DBHelper.ExecuteQuery(
    "DELETE FROM Children
WHERE ChildID = @id",
    new SqlParameter[] { new
SqlParameter("@id", selectedChildID)
});

        if (result)
        {
            MessageBox.Show("Deleted
Successfully ");
            ClearForm();
            LoadChildren();
        }
    }

    // Open the detailed child profile
    window
    private void
    btnViewProfile_Click(object sender,
    EventArgs e)
    {
        if (selectedChildID == 0)
        {
            MessageBox.Show("Please
select a child!");
            return;
        }

        // Create the profile form with all
        3 required values
        frmChildProfile f = new
        frmChildProfile(selectedChildID,
        userRole, userID);
        f.Show();

```

```

        // Only hide this form if profile
        form is actually visible and open
        if (f.Visible)
        {
            this.Hide();
        }
    }

    // Reset all form inputs to default
    blank entries
    private void ClearForm()
    {
        txtChildName.Clear();
        numAge.Value = 1;
        txtLocation.Clear();
        cmbGender.SelectedIndex = 0;
        cmbStatus.SelectedIndex = 0;
        selectedChildID = 0;
    }

    // Close form and reveal the hidden
    system user dashboard
    private void btnHome_Click(object
    sender, EventArgs e)
    {
        foreach (Form openForm in
        Application.OpenForms)
        {
            if (openForm is
            frmAdminDashboard || openForm is
            frmStaffDashboard)
            {
                openForm.Show();
                break;
            }
        }
    }

```

```

        this.Close();
    }

    // Confirm prompt and route user
    safely back to login screen
    private void
    btnLogout_Click(object sender,
    EventArgs e)
    {
        DialogResult result =
    MessageBox.Show(
        "Are you sure to logout?",
    "Logout",
        MessageBoxButtons.YesNo,
    MessageBoxIcon.Question);

        if (result == DialogResult.Yes)
        {
            frmLogin login = new
    frmLogin();
            login.Show();
            this.Close();
        }
    }

    // Navigate back to the home user
    dashboard form screen
    private void btnBack_Click(object
    sender, EventArgs e)
    {
        foreach (Form openForm in
    Application.OpenForms)

```

```

    {
        if (openForm is
    frmAdminDashboard || openForm is
    frmStaffDashboard)
        {
            openForm.Show();
            break;
        }
    }
    this.Close();
}

// Public method to refresh data
table listings externally
public void RefreshChildList()
{
    LoadChildren();
    ClearForm();
}

// Reset all current values in the
form inputs and reload table list
private void btnClear_Click(object
sender, EventArgs e)
{
    ClearForm();
    LoadChildren();
}
}
}

```

5. Orphanage Management Module:

File Name: [frmOrphanage.cs](#)

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace
StreetChildrenCareSystem.Forms
{
    public partial class frmOrphanage :
    Form
    {
        // Global variables for user tracking
        and selection
        private string userRole;
        private string userID;
        private int selectedOrpID = 0;

        // Constructor: receives role and
        userID from dashboard
        public frmOrphanage(string role,
        string uid)
        {
            InitializeComponent();
            userRole = role;
            userID = uid;
        }

        // Setup form and load data when
        window opens
        private void
        frmOrphanage_Load(object sender,
        EventArgs e)
        {
            SetPermissions();
            LoadOrphanages();

            dgvOrphanage.AutoSizeColumnsMode =
            DataGridViewAutoSizeColumnsMode.Fi
            ll;

            // Add sort options to combo box
            cmbSort.Items.Add("Sort by
            Name");
            cmbSort.Items.Add("Sort by
            Location");
            cmbSort.SelectedIndex = 0;
        }

        // Role-based Access Control:
        Admin sees all buttons; Staff gets
        view-only
        private void SetPermissions()
        {
            if (userRole == "Staff")
            {
                btnSave.Visible = false;
                btnUpdate.Visible = false;
                btnDelete.Visible = false;
                btnClear.Visible = false;
                txtOrpName.ReadOnly = true;
            }
        }
    }
}
```

```

        txtLocation.ReadOnly = true;
        txtONumber.ReadOnly = true;
    }
}

// Fetch all orphanage records from
// database to grid view
private void LoadOrphanages()
{
    string query =
        "SELECT OrpID, OrpName,
        OrpLocation, OrpNumber" +
        " FROM Orphanages ORDER
        BY OrpName ASC";

    DataTable dt =
        DBHelper.GetData(query);
    dgvOrphanage.DataSource = dt;
}

// Validate and insert a new
// orphanage record
private void btnSave_Click(object
sender, EventArgs e)
{
    // Input validations
    if
(string.IsNullOrEmpty(txtOrpName
.Text))
    {
        MessageBox.Show("Orphanage Name is
        required!", "Validation",
        MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        txtOrpName.Focus();
        return;
    }
}

```

```

    }

    if
(string.IsNullOrEmpty(txtLocation.
Text))
    {
        MessageBox.Show("Location
        is required!", "Validation",
        MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        txtLocation.Focus();
        return;
    }

    if
(string.IsNullOrEmpty(txtONumbe
r.Text))
    {
        MessageBox.Show("Contact
        Number is required!", "Validation",
        MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        txtONumber.Focus();
        return;
    }

    // Database insertion logic
    string query =
        "INSERT INTO Orphanages
        (OrpName, OrpLocation, OrpNumber)"
        +
        " VALUES (@name, @loc,
        @num)";

    SqlParameter[] p =
    {

```

```

        new SqlParameter("@name",
txtOrpName.Text.Trim()),
        new SqlParameter("@loc",
txtLocation.Text.Trim()),
        new SqlParameter("@num",
txtONumber.Text.Trim())
    };

    if
(DBHelper.ExecuteQuery(query, p))
    {

MessageBox.Show("Orphanage Saved
, "Success",
        MessageBoxButtons.OK,
MessageBoxIcon.Information);
        ClearForm();
        LoadOrphanages();
    }
}

// Fill form fields automatically
when clicking a row in the grid view
private void
dgvOrphanage_CellClick(object sender,
DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row =
dgvOrphanage.Rows[e.RowIndex];
        selectedOrpID =
Convert.ToInt32(row.Cells["OrpID"].Value);
        txtOrpName.Text =
row.Cells["OrpName"].Value.ToString();

```

```

        txtLocation.Text =
row.Cells["OrpLocation"].Value.ToString();
        txtONumber.Text =
row.Cells["OrpNumber"].Value.ToString();
    }
}

// Update details of the selected
orphanage
private void
btnUpdate_Click(object sender,
EventArgs e)
{
    if (selectedOrpID == 0)
    {
        MessageBox.Show("Please
select an Orphanage first!");
        return;
    }

    string query =
        "UPDATE Orphanages SET
OrpName = @name," +
        " OrpLocation = @loc,
OrpNumber = @num" +
        " WHERE OrpID = @id";

    SqlParameter[] p =
    {
        new SqlParameter("@name",
txtOrpName.Text.Trim()),
        new SqlParameter("@loc",
txtLocation.Text.Trim()),
        new SqlParameter("@num",
txtONumber.Text.Trim()),

```

```

        new SqlParameter("@id",
selectedOrpID)
        };

        if
(DBHelper.ExecuteQuery(query, p))
        {
            MessageBox.Show("Updated
Successfully ");
            ClearForm();
            LoadOrphanages();
        }
    }
}

```

```

// Delete the selected orphanage
after confirmation
private void btnDelete_Click(object
sender, EventArgs e)
{
    if (selectedOrpID == 0)
    {
        MessageBox.Show("Please
select an Orphanage first!");
        return;
    }
}

```

```

        DialogResult dr =
MessageBox.Show(
    "Are you sure to delete this
Orphanage?", "Confirm Delete",
    MessageBoxButtons.YesNo,
    MessageBoxIcon.Warning);

```

```

        if (dr == DialogResult.Yes)
        {
            if (DBHelper.ExecuteQuery(

```

```

        "DELETE FROM
Orphanages WHERE OrpID = @id",
        new SqlParameter[] { new
SqlParameter("@id", selectedOrpID) }))
        {
            MessageBox.Show("Deleted
Successfully ");
            ClearForm();
            LoadOrphanages();
        }
    }
}

```

```

// Search orphanages using name or
location keywords

```

```

private void btnSearch_Click(object
sender, EventArgs e)
{
    string keyword =
txtSearch.Text.Trim();

```

```

        string query =
        "SELECT OrpID, OrpName,
OrpLocation, OrpNumber" +
        " FROM Orphanages" +
        " WHERE OrpName LIKE
@kw OR OrpLocation LIKE @kw";

```

```

        SqlParameter[] p =
        {
            new SqlParameter("@kw",
"%" + keyword + "%")
        };

```

```

        DataTable dt =
DBHelper.GetData(query, p);
dgvOrphanage.DataSource = dt;

```

```

    }

    // Handle database sorting according
    to dropdown selection
    private void
    cmbSort_SelectedIndexChanged(object
    sender, EventArgs e)
    {
        string sortBy = "";

        if (cmbSort.Text == "Sort by
        Name")
            sortBy = "OrpName ASC";
        else if (cmbSort.Text == "Sort by
        Location")
            sortBy = "OrpLocation ASC";

        if (sortBy == "") return;

        DataTable dt =
        DBHelper.GetData(
            "SELECT OrpID, OrpName,
            OrpLocation, OrpNumber" +
            " FROM Orphanages ORDER
            BY " + sortBy);

        dgvOrphanage.DataSource = dt;
    }

    // Reset all text boxes and variable
    parameters
    private void ClearForm()
    {
        txtOrpName.Clear();
        txtLocation.Clear();
        txtONumber.Clear();
        txtSearch.Clear();

```

```

        selectedOrpID = 0;
    }

    // Clear button click action
    private void btnClear_Click(object
    sender, EventArgs e)
    {
        ClearForm();
        LoadOrphanages();
    }

    // Navigate back to the home
    dashboard screen
    private void btnHome_Click(object
    sender, EventArgs e)
    {
        foreach (Form openForm in
        Application.OpenForms)
        {
            if (openForm is
            frmAdminDashboard || openForm is
            frmStaffDashboard)
            {
                openForm.Show();
                break;
            }
        }
        this.Close();
    }

    // Navigate to the previous
    dashboard screen
    private void btnBack_Click(object
    sender, EventArgs e)
    {
        foreach (Form openForm in
        Application.OpenForms)

```

```

        {
            if (openForm is
frmAdminDashboard || openForm is
frmStaffDashboard)
            {
                openForm.Show();
                break;
            }
        }
        this.Close();
    }

    // Process secure user logout and
    redirect to login page
    private void
    btnLogout_Click(object sender,
    EventArgs e)
    {
        DialogResult result =
    MessageBox.Show(
        "Are you sure to logout?",
    "Logout",
        MessageBoxButtons.YesNo,
    MessageBoxIcon.Question);

```

```

        if (result == DialogResult.Yes)
        {
            Form dashboardForm = null;
            foreach (Form openForm in
    Application.OpenForms)
            {
                if (openForm is
    frmAdminDashboard || openForm is
    frmStaffDashboard)
                    dashboardForm =
    openForm;
            }
            if (dashboardForm != null)
    dashboardForm.Close();

            frmLogin login = new
    frmLogin();
            login.Show();
            this.Close();
        }
    }
}

```

6. Foundation Management Module:

File Name: [frmFoundation.cs](#)

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace
StreetChildrenCareSystem.Forms
{
    public partial class frmFoundation :
    Form
    {
        // Internal variables tracking system
        actions
        private string userRole;
        private string userID;
        private int selectedFouID = 0;

        // Constructor initialization
        public frmFoundation(string role,
        string uid)
        {
            InitializeComponent();
            userRole = role;
            userID = uid;
        }

        // Populate controls and
        configurations on window load
```

```
        private void
        frmFoundation_Load(object sender,
        EventArgs e)
        {
            SetPermissions();
            LoadFoundations();

            // Fix: make columns fill the full
            width of the DataGridView

            dgvFoundation.AutoSizeColumnsMode
            =

            DataGridViewAutoSizeColumnsMode.Fi
            ll;

            // SupportType ComboBox

            cmbSupportType.Items.Add("Food");

            // SupportType ComboBox

            cmbSupportType.Items.Add("Food");
            cmbSupportType.Items.Add("Clothing");
            cmbSupportType.Items.Add("Education"
            );
            cmbSupportType.Items.Add("Medical");
            cmbSupportType.SelectedIndex =
            0;
```

```

        // Sort ComboBox
        cmbSort.Items.Add("Sort by
Name");
        cmbSort.Items.Add("Sort by
Type");
        cmbSort.SelectedIndex = 0;
    }

    // Limit data manipulation options
    for non-administrator profiles
    private void SetPermissions()
    {
        if (userRole == "Staff")
        {
            btnSave.Visible = false;
            btnUpdate.Visible = false;
            btnDelete.Visible = false;
            btnClear.Visible = false;
            txtFouName.ReadOnly = true;
            txtFNumber.ReadOnly = true;
            cmbSupportType.Enabled =
false;
        }
    }

    // Query database to list all
    foundations
    private void LoadFoundations()
    {
        DataTable dt =
DBHelper.GetData(
        "SELECT FouID, FouName,"
+
        " SupportType, FouNumber" +
        " FROM Foundations" +
        " ORDER BY FouName
ASC");

```

```

        dgvFoundation.DataSource = dt;
    }

    // Collect parameters and execute
    entry logic for new organizations
    private void btnSave_Click(object
sender, EventArgs e)
    {
        if
(string.IsNullOrEmpty(txtFouName
.Text))
        {
            MessageBox.Show("Foundation Name
required!");
            txtFouName.Focus();
            return;
        }

        if
(string.IsNullOrEmpty(txtFNumber
.Text))
        {
            MessageBox.Show("Contact
Number required!");
            txtFNumber.Focus();
            return;
        }

        string query =
        "INSERT INTO Foundations"
+
        " (FouName, SupportType," +
        " FouNumber)" +
        " VALUES (@name, @type,
@num)";

```

```

        SqlParameter[] p =
        {
            new SqlParameter("@name",
txtFouName.Text.Trim()),
            new SqlParameter("@type",
cmbSupportType.Text),
            new SqlParameter("@num",
txtFNumber.Text.Trim())
        };

        if
(DBHelper.ExecuteQuery(query, p))
        {

MessageBox.Show("Foundation Saved
");

            ClearForm();
            LoadFoundations();
        }
    }

    // Map data from UI table cell back
into editing fields
    private void
dgvFoundation_CellClick(object sender,
DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            DataGridViewRow row =
dgvFoundation.Rows[e.RowIndex];

            selectedFouID =
Convert.ToInt32(row.Cells["FouID"].Val
ue);

            txtFouName.Text =
row.Cells["FouName"].Value.ToString();

```

```

        cmbSupportType.Text =
row.Cells["SupportType"].Value.ToStrin
g();

        txtFNumber.Text =
row.Cells["FouNumber"].Value.ToString
();
    }
}

// Handle modification queries for
existing foundations
    private void
btnUpdate_Click(object sender,
EventArgs e)
    {
        if (selectedFouID == 0)
        {
            MessageBox.Show("Please
select a Foundation!");
            return;
        }

        string query =
            "UPDATE Foundations SET"
+
            " FouName = @name," +
            " SupportType = @type," +
            " FouNumber = @num" +
            " WHERE FouID = @id";

        SqlParameter[] p =
        {
            new SqlParameter("@name",
txtFouName.Text.Trim()),
            new SqlParameter("@type",
cmbSupportType.Text),

```

```

        new SqlParameter("@num",
txtFNumber.Text.Trim()),
        new SqlParameter("@id",
selectedFouID)
    };

    if
(DBHelper.ExecuteQuery(query, p))
    {
        MessageBox.Show("Updated
Successfully ");
        ClearForm();
        LoadFoundations();
    }
}

// Permanently drop an institutional
reference from storage
private void btnDelete_Click(object
sender, EventArgs e)
{
    if (selectedFouID == 0)
    {
        MessageBox.Show("Please
select a Foundation!");
        return;
    }

    DialogResult dr =
MessageBox.Show(
    "Are you sure to delete?",
    "Confirm",
    MessageBoxButtons.YesNo,
    MessageBoxIcon.Warning);

    if (dr == DialogResult.Yes)
    {

```

```

        if (DBHelper.ExecuteQuery(
            "DELETE FROM
Foundations WHERE FouID = @id",
            new SqlParameter[] { new
SqlParameter("@id", selectedFouID) }))
        {
            MessageBox.Show("Deleted
Successfully ");
            ClearForm();
            LoadFoundations();
        }
    }
}

// Query database filtering records
matching standard string metrics
private void btnSearch_Click(object
sender, EventArgs e)
{
    string kw =
txtSearch.Text.Trim();
    SqlParameter[] p =
    {
        new SqlParameter("@kw",
"%" + kw + "%")
    };

    DataTable dt =
DBHelper.GetData(
    "SELECT FouID, FouName,"
+
    " SupportType, FouNumber" +
    " FROM Foundations" +
    " WHERE FouName LIKE
@kw" +
    " OR SupportType LIKE
@kw", p);

```

```

        dgvFoundation.DataSource = dt;
    }

    // Sort execution routing according
    to configuration selections
    private void
    cmbSort_SelectedIndexChanged(object
    sender, EventArgs e)
    {
        string s = cmbSort.Text == "Sort
    by Name" ? "FouName ASC" :
    "SupportType ASC";

        DataTable dt =
    DBHelper.GetData(
        "SELECT FouID, FouName,"
    +
        " SupportType, FouNumber" +
        " FROM Foundations ORDER
    BY " + s);
        dgvFoundation.DataSource = dt;
    }

    // Reinitialize layout tracking
    indices and view state text blocks
    private void ClearForm()
    {
        txtFouName.Clear();
        txtFNumber.Clear();
        txtSearch.Clear();
        cmbSupportType.SelectedIndex =
    0;
        selectedFouID = 0;
    }

    // Form clearing trigger interface

```

```

        private void btnClear_Click(object
    sender, EventArgs e)
        {
            ClearForm();
            LoadFoundations();
        }

        // Return session views to preceding
        dashboard forms
        private void btnBack_Click(object
    sender, EventArgs e)
        {
            foreach (Form openForm in
    Application.OpenForms)
            {
                if (openForm is
    frmAdminDashboard || openForm is
    frmStaffDashboard)
                {
                    openForm.Show();
                    break;
                }
            }
            this.Close();
        }

        // Redirect application focus back to
        central screen layout structures
        private void btnHome_Click(object
    sender, EventArgs e)
        {
            foreach (Form openForm in
    Application.OpenForms)
            {
                if (openForm is
    frmAdminDashboard || openForm is
    frmStaffDashboard)

```

```

        {
            openForm.Show();
            break;
        }
    }
    this.Close();
}

// Kill relevant current views and
restore central authentication dialogue
access
private void
btnLogout_Click(object sender,
EventArgs e)
{
    DialogResult result =
    MessageBox.Show(

```

```

        "Are you sure to logout?",
        "Logout",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

        if (result == DialogResult.Yes)
        {
            frmLogin login = new
            frmLogin();
            login.Show();
            this.Close();
        }
    }
}

```

7. Vaccination Tracking Module:

File Name: [frmVaccination.cs](#)

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace StreetChildrenCareSystem.Forms
{
    public partial class frmVaccination :
    Form
    {
        // Track state context parameters
        private string userRole;
        private int selectedVacID = 0;

        // Constructor to store execution
        authorizations
        public frmVaccination(string role)
        {
            InitializeComponent();
            userRole = role;
        }

        // Load visual drops and lists
        required for medication processing
        private void
        frmVaccination_Load(object sender,
        EventArgs e)
        {
            SetPermissions();
            LoadChildCombo();
            LoadAllVaccinations();

            dgvVaccination.AutoSizeColumnsMode
            =
            DataGridViewAutoSizeColumnsMode.Fi
            ll;

            // Add clinical target options

            cmbVacName.Items.Add("BCG");

            cmbVacName.Items.Add("Polio");

            cmbVacName.Items.Add("DPT");

            cmbVacName.Items.Add("Measles");

            cmbVacName.Items.Add("Hepatitis B");
            cmbVacName.SelectedIndex = 0;

            // Setup programmatic state flags

            cmbVacStatus.Items.Add("Pending");

            cmbVacStatus.Items.Add("Completed");

            cmbVacStatus.Items.Add("Overdue");
            cmbVacStatus.SelectedIndex = 0;
        }
    }
}
```

```

        // Setup relational view filtering
        configurations
        cmbSort.Items.Add("Sort by
        Date");
        cmbSort.Items.Add("Sort by
        Status");
        cmbSort.Items.Add("Sort by
        Child Name");
        cmbSort.SelectedIndex = 0;
    }

```

```

    // Apply strict application
    restrictions to Staff users
    private void SetPermissions()
    {
        if (userRole == "Staff")
        {
            btnSave.Visible = false;
            btnUpdate.Visible = false;
            btnDelete.Visible = false;
            btnClear.Visible = false;
            cmbChild.Enabled = false;
            cmbVacName.Enabled = false;
            dtpGivenDate.Enabled = false;
            dtpNextDueDate.Enabled =
false;
            cmbVacStatus.Enabled = false;
        }
    }

```

```

    // Populates relational child tracking
    keys inside selection dropdowns
    private void LoadChildCombo()
    {
        DataTable dt =
        DBHelper.GetData(

```

```

        "SELECT ChildID,
        ChildName FROM Children ORDER
        BY ChildName ASC");
        cmbChild.DisplayMember =
        "ChildName";
        cmbChild.ValueMember =
        "ChildID";
        cmbChild.DataSource = dt;
    }

```

```

    // Join operational system tables to
    present structured history listings
    private void LoadAllVaccinations()
    {
        string query =
            "SELECT V.VacID,
            C.ChildName, V.VacName," +
            " V.GivenDate,
            V.NextDueDate, V.VacStatus" +
            " FROM Vaccinations V" +
            " JOIN Children C ON
            V.ChildID = C.ChildID" +
            " ORDER BY V.GivenDate
            DESC";

        DataTable dt =
        DBHelper.GetData(query);
        dgvVaccination.DataSource = dt;
    }

```

```

    // Parse metrics to secure an updated
    item event tracking row entry
    private void btnSave_Click(object
    sender, EventArgs e)
    {
        if (cmbChild.SelectedValue ==
        null)

```

```

        {
            MessageBox.Show("Please
select a Child!");
            return;
        }

        string query =
            "INSERT INTO Vaccinations"
+
            " (ChildID, VacName,
GivenDate, NextDueDate, VacStatus)" +
            " VALUES (@cid, @vname,
@gdate, @ndate, @vstatus)";

        SqlParameter[] p =
        {
            new SqlParameter("@cid",
cmbChild.SelectedValue),
            new SqlParameter("@vname",
cmbVacName.Text),
            new SqlParameter("@gdate",
dtpGivenDate.Value.ToString("yyyy-M
M-dd")),
            new SqlParameter("@ndate",
dtpNextDueDate.Value.ToString("yyyy-
MM-dd")),
            new SqlParameter("@vstatus",
cmbVacStatus.Text)
        };

        if
(DBHelper.ExecuteQuery(query, p))
        {

            MessageBox.Show("Vaccination Record
Saved ");
            ClearForm();

```

```

            LoadAllVaccinations();
        }
    }

    // Read specific row selection states
safely with error protection
    private void
dgvVaccination_CellClick(object sender,
DataGridViewCellEventArgs e)
    {
        // e.RowIndex -1 means header
was clicked, so ignore it
        if (e.RowIndex < 0) return;

        try
        {
            DataGridViewRow row =
dgvVaccination.Rows[e.RowIndex];

            // Store selected VacID safely
            if (row.Cells["VacID"].Value
!= null && row.Cells["VacID"].Value !=
DBNull.Value)
                selectedVacID =
Convert.ToInt32(row.Cells["VacID"].Val
ue);

            // Fill form fields from selected
row
            cmbVacName.Text =
row.Cells["VacName"].Value.ToString();
            cmbVacStatus.Text =
row.Cells["VacStatus"].Value.ToString();

            if
(row.Cells["GivenDate"].Value !=
DBNull.Value)

```

```

        dtpGivenDate.Value =
Convert.ToDateTime(row.Cells["GivenD
ate"].Value);

```

```

        if
(row.Cells["NextDueDate"].Value !=
DBNull.Value)
            dtpNextDueDate.Value =
Convert.ToDateTime(row.Cells["NextDu
eDate"].Value);
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error
reading row: " + ex.Message);
        }
    }

```

```

        // Manage record adjustment and
update routing parameters safely
        private void
btnUpdate_Click(object sender,
EventArgs e)
        {
            // Read directly from the selected
row - more reliable than stored variable
            if (dgvVaccination.CurrentRow
== null)
            {
                MessageBox.Show("Please
select a record first!");
                return;
            }

```

```

        // Get VacID directly from the
currently highlighted row

```

```

        selectedVacID =
Convert.ToInt32(dgvVaccination.Current
Row.Cells["VacID"].Value);

```

```

        string query =
            "UPDATE Vaccinations SET"
+
            " VacName = @vname," +
            " GivenDate = @gdate," +
            " NextDueDate = @ndate," +
            " VacStatus = @vstatus" +
            " WHERE VacID = @id";

        SqlParameter[] p =
        {
            new SqlParameter("@vname",
cmbVacName.Text),
            new SqlParameter("@gdate",
dtpGivenDate.Value.ToString("yyyy-M
M-dd")),
            new SqlParameter("@ndate",
dtpNextDueDate.Value.ToString("yyyy-
MM-dd")),
            new SqlParameter("@vstatus",
cmbVacStatus.Text),
            new SqlParameter("@id",
selectedVacID)
        };

```

```

        if
(DBHelper.ExecuteQuery(query, p))
        {
            MessageBox.Show("Updated
Successfully ");
            ClearForm();
            LoadAllVaccinations();
        }

```

```

    }

    // Intercept action events to safely
    remove clinical tracking logs
    private void btnDelete_Click(object
sender, EventArgs e)
    {
        // Read directly from the selected
row - more reliable than stored variable
        if (dgvVaccination.CurrentRow
== null)
        {
            MessageBox.Show("Please
select a record first!");
            return;
        }

        // Get VacID directly from the
currently highlighted row
        selectedVacID =
Convert.ToInt32(dgvVaccination.Current
Row.Cells["VacID"].Value);

        DialogResult dr =
MessageBox.Show(
            "Delete this vaccination
record?", "Confirm",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Warning);

        if (dr == DialogResult.Yes)
        {
            if (DBHelper.ExecuteQuery(
                "DELETE FROM
Vaccinations WHERE VacID = @id",
                new SqlParameter[] { new
SqlParameter("@id", selectedVacID) }))

```

```

        {
            MessageBox.Show("Deleted
Successfully ");
            ClearForm();
            LoadAllVaccinations();
        }
    }

    // Perform complex string matches
across name keys or status identifiers
    private void btnSearch_Click(object
sender, EventArgs e)
    {
        string kw =
txtSearch.Text.Trim();

        string query =
            "SELECT V.VacID,
            C.ChildName, V.VacName," +
            " V.GivenDate,
            V.NextDueDate, V.VacStatus" +
            " FROM Vaccinations V" +
            " JOIN Children C ON
            V.ChildID = C.ChildID" +
            " WHERE C.ChildName LIKE
            @kw" +
            " OR V.VacName LIKE @kw
            OR V.VacStatus LIKE @kw";

        SqlParameter[] p = { new
SqlParameter("@kw", "%" + kw + "%")
};

        DataTable dt =
DBHelper.GetData(query, p);
        dgvVaccination.DataSource = dt;
    }

```

```

        // Organize visual table items based
on date components or statuses
        private void
cmbSort_SelectedIndexChanged(object
sender, EventArgs e)
        {
            string sortBy = "";

            if (cmbSort.Text == "Sort by
Date")
                sortBy = "V.GivenDate
DESC";
            else if (cmbSort.Text == "Sort by
Status")
                sortBy = "V.VacStatus ASC";
            else if (cmbSort.Text == "Sort by
Child Name")
                sortBy = "C.ChildName ASC";

            if (sortBy == "") return;

            DataTable dt =
DBHelper.GetData(
                "SELECT V.VacID,
C.ChildName, V.VacName," +
                " V.GivenDate,
V.NextDueDate, V.VacStatus" +
                " FROM Vaccinations V" +
                " JOIN Children C ON
V.ChildID = C.ChildID" +
                " ORDER BY " + sortBy);

            dgvVaccination.DataSource = dt;
        }

```

```

        // Clear tracking structures and reset
system clock markers on view fields
        private void ClearForm()
        {
            cmbVacName.SelectedIndex = 0;
            cmbVacStatus.SelectedIndex = 0;
            dtpGivenDate.Value =
DateTime.Now;
            dtpNextDueDate.Value =
DateTime.Now;
            selectedVacID = 0;
        }

        // Form fields clear trigger action
        private void btnClear_Click(object
sender, EventArgs e)
        {
            ClearForm();
            LoadAllVaccinations();
        }

        // Relocate dashboard display
frameworks back into focus
        private void btnHome_Click(object
sender, EventArgs e)
        {
            foreach (Form openForm in
Application.OpenForms)
            {
                if (openForm is
frmAdminDashboard || openForm is
frmStaffDashboard)
                {
                    openForm.Show();
                    break;
                }
            }
        }

```

```

        this.Close();
    }

    // Roll views back to previous steps
    on form structure trees
    private void btnBack_Click(object
sender, EventArgs e)
    {
        foreach (Form openForm in
Application.OpenForms)
        {
            if (openForm is
frmAdminDashboard || openForm is
frmStaffDashboard)
            {
                openForm.Show();
                break;
            }
        }
        this.Close();
    }

    // Confirm identity changes and
    close main background threads to return
    to login screen
    private void
btnLogout_Click(object sender,
EventArgs e)
    {
        DialogResult result =
MessageBox.Show(
            "Are you sure to logout?",
            "Logout",

```

```

        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

        if (result == DialogResult.Yes)
        {
            Form dashboardForm = null;
            foreach (Form openForm in
Application.OpenForms)
            {
                if (openForm is
frmAdminDashboard || openForm is
frmStaffDashboard)
                    dashboardForm =
openForm;
            }
            if (dashboardForm != null)
                dashboardForm.Close();

            frmLogin login = new
frmLogin();
            login.Show();
            this.Close();
        }
    }

    private void
cmbVacStatus_SelectedIndexChanged(o
bject sender, EventArgs e)
    {
    }
}

```

CONCLUSION

Project Summary & Goals Achieved

The Street Children Care System was successfully developed and tested as a fully functional C# (.NET Windows Forms) desktop application with a SQL Server (SSMS) back-end. All defined project objectives were met - including secure role-based login, complete CRUD operations across all five database tables, real-time dashboard statistics, and a Staff-restricted view-only mode. The application runs stably in the Visual Studio environment, and the database connection through ADO.NET with parameterized queries performs reliably and securely. This project effectively replaces inefficient manual records with a structured, digital, and access-controlled management system.

System Limitations & Future Scope

- **Current Limitation:** The system is desktop-only with no remote access, and passwords are stored without encryption.
- **Future Enhancement 1:** Migrate the database to a cloud platform such as Microsoft Azure SQL and develop a web or mobile version for multi-user remote access.
- **Future Enhancement 2:** Add automated email or SMS alerts for overdue vaccinations and implement PDF report generation for child and vaccination records.

Final Takeaway

The Street Children Care System proves that focused, well-structured software can deliver real value in managing vulnerable populations efficiently. This project establishes a solid technical foundation that can be expanded into a more advanced, cloud-ready, and feature-rich system in future development cycles.

GitHub Link

<https://github.com/ismmmx/StreetChildrenCareSystem/>

REFERENCES

1. Microsoft Corporation. (2024). *C# Language Documentation — C# Guide*. Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/>
2. Microsoft Corporation. (2024). *Windows Forms for .NET 6 — Desktop Application Development*. Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>
3. Microsoft Corporation. (2024). *Visual Studio 2022 IDE Documentation — Develop Apps for Windows*. Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/visualstudio/windows/>
4. Troelsen, A. & Japikse, P. (2022). *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming* (10th ed.). Apress, New York. ISBN: 978-1484278697.
5. Microsoft Corporation. (2024). *SQL Server Management Studio (SSMS) — Official Documentation*. Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/sql/ssms/>
6. Microsoft Corporation. (2024). *Transact-SQL Reference (T-SQL) — SQL Server Documentation*. Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/sql/t-sql/language-reference>
7. Elmasri, R. & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education, New York. ISBN: 978-0133970777.
8. Osei, E., Addo, P. & Mensah, J. (2021). *Design and Implementation of a Child Health Record Management System for Rural Communities*. International Journal of Computer Applications, 183(12), pp. 1–8. Available at: <https://www.ijcaonline.org/>
9. Sharma, R. & Singh, P. (2022). *A Desktop-Based Orphan Management System Using C# and SQL Server for NGO Data Tracking*. ResearchGate. Available at: <https://www.researchgate.net/publication/> (Search: "Orphan Management System C# SQL Server")
10. Hossain, M. T. & Karim, A. R. (2021). *Development of a Digital Child Registration and Tracking System for Non-Governmental Organizations in Developing Countries*. Asian Journal of Computer Science and Information Technology, 11(3), pp. 22–34. Available at: <https://www.researchgate.net/> (Search: "Child Registration Tracking System NGO")
11. Alam, M. S. & Begum, R. (2020). *Computerized Health Record System for Street Children: A Case Study of NGO Operations in Bangladesh*. International Journal of

Advanced Computer Science and Applications (IJACSA), 11(8), pp. 55–63. Available at: <https://thesai.org/Publications/IJACSA>

12. Patel, N., Shah, D. & Desai, M. (2022). *Role-Based Access Control in Desktop Applications: Implementation Using Windows Forms and SQL Server*. International Journal of Engineering Research and Technology (IJERT), 11(4), pp. 112–118. Available at: <https://www.ijert.org/>

13. UNICEF Bangladesh. (2023). *Children in Bangladesh — Situation and Key Statistics on Child Protection*. UNICEF. Available at: <https://www.unicef.org/bangladesh/children-bangladesh>

14. Bangladesh Bureau of Statistics (BBS). (2022). *Child Poverty and Deprivation in Bangladesh — Multiple Indicator Cluster Survey (MICS)*. Government of Bangladesh. Available at: <https://bbs.gov.bd/>

15. Terre des hommes Foundation. (2022). *Street Children: A Global Perspective on Child Protection and Registration Systems*. Tdh Report, Lausanne, Switzerland. Available at: <https://www.tdh.org/>